

CyberSafe

An AI-Powered Cybersecurity Assistant for Everyone

Technical Requirements Document (TRD)

Version: 1.0

Status: Draft

Owner: Engineering Team

Companion document to: CyberSafe PRD v1.0. This TRD defines how each PRD requirement is technically implemented: architecture, data contracts, APIs, algorithms, and non-functional requirements.

Table of Contents

1. Scope
2. Technology Stack
3. System Architecture
4. Module-Level Technical Specifications
 - 4.1 Input Processor
 - 4.2 Security Engines
 - 4.3 Risk Engine
 - 4.4 LLM Explanation Layer
 - 4.5 Action Plan Generator
5. Data Contracts (Schemas)
6. API Specification
7. Risk Scoring Logic
8. Non-Functional Requirements
9. Security & Privacy Requirements
10. Error Handling
11. Deployment Architecture

1. Scope

This TRD covers the technical implementation of every component defined in the CyberSafe PRD: the four input types, the five-stage processing pipeline (Input Processor → Security Engines → Risk Engine → LLM Explanation Layer → Action Plan), the user preference system, and the eight threat categories. No feature outside the PRD scope is included.

2. Technology Stack

Layer	Technology / Approach
Frontend	Web app (React) — upload UI, preference selectors, tabbed risk output
Backend / API	REST API service (Python — FastAPI) orchestrating the pipeline
Vision / OCR	Claude (vision-capable model) — extracts text and visual signals directly from screenshots; replaces a separate OCR engine
URL / Domain Analysis	URL parser (Python) + WHOIS lookup + SSL certificate check
Rules Engine	Deterministic pattern/signature matcher (regex + keyword/heuristic ruleset)
ML Model	Supervised classifier (e.g. gradient-boosted trees) trained on lexical URL features, domain age, SSL status, and content features
LLM Explanation Layer	Claude (text model) — converts structured risk JSON into Simple / Detailed / Technical explanations
Data Storage	Relational DB (submissions, risk results) + object storage (uploaded screenshots/files, short retention)
Deployment	Containerized services (Docker), deployed behind an API gateway

3. System Architecture

Request-level data flow through the system:

```
Client (Web/App)
| POST /api/v1/analyze { input_type, payload, preferences }
v
API Gateway → Analysis Orchestrator
|
+--> [1] Input Processor
|     screenshot → Vision/OCR (Claude vision)
|     url        → URL Parser (domain, SSL, WHOIS)
|     text       → Text Parser (structure, headers, links)
|     file       → File Parser (metadata, attachment scan)
|     output     → normalized_content (see Sec.5 InputProcessorOutput)
|
+--> [2] Security Engines (parallel execution)
|     Rules Engine → signature_matches[]
|     ML Model    → ml_classification { label, probability }
|     Security Checks → check_results[] (domain/SSL/sender/attachment)
|
+--> [3] Risk Engine
|     combines [2] outputs → RiskAssessment JSON (Sec.5)
|
+--> [4] LLM Explanation Layer
|     RiskAssessment + user_preferences → Explanation (Simple/Detailed/Technical)
|
+--> [5] Action Plan Generator
|     RiskAssessment → { what_happened, why_risky, what_to_do[] }
|
v
Response Composer → JSON response → Client renders Analysis Output UI
```

4. Module-Level Technical Specifications

4.1 Input Processor

Sub-modules	OCR (image), URL Parser, Text Parser, File Parser
OCR	Vision-capable LLM call on uploaded image; extracts raw text + visual red flags (spoofed logos, mismatched sender styling)
URL Parser	Extracts domain, subdomain, path, query params, SSL certificate validity, domain registration age (WHOIS)
Text Parser	Extracts sender, subject, links, and body content from pasted email/message text
File Parser	Extracts file type, hash, and metadata for optional document/executable uploads
Output contract	InputProcessorOutput JSON — see Section 5
Failure mode	If extraction confidence is low (e.g. blurry image), return a low_confidence flag and request re-upload

4.2 Security Engines

Approach	Deterministic signature/pattern matching — regex + keyword heuristics
Signals checked	Urgency phrases, credential/OTP requests, mismatched sender domain, shortened/obfuscated URLs, known malicious domain list
Output	List of matched_signals with a fixed severity weight per rule

Task	Binary/multi-class classification: benign vs. phishing / malware / social engineering
Features	URL lexical features (length, special chars, entropy), domain age, SSL validity, content keyword vectors
Output	{ label, probability }
Retraining	Periodic retraining on labeled submission data with human-reviewed corrections

Checks performed	Domain reputation, SSL certificate validity, sender authenticity (SPF/DKIM where available), attachment type/hash scan
Output	List of check_results, each pass/fail with a reason string

4.3 Risk Engine

Function	Combines Rules Engine, ML Model, and Security Checks outputs into one verdict
Outputs	threat_detection (type), severity_assessment (Low/Medium/High), evidence_collection (reasons/indicators), confidence score
Scoring	Weighted combination of rule matches + ML probability + failed security checks — see Section 7

4.4 LLM Explanation Layer

Function	Converts RiskAssessment JSON into natural-language explanation — never re-decides risk, only explains it
Input	RiskAssessment JSON + user_preferences (who_are_you, technical_level, focus_area)
Output styles	Simple (non-technical), Detailed (balanced), Technical (advanced) — all three generated per request
Constraint	Must be grounded strictly in the evidence_collection field; no unsupported claims

4.5 Action Plan Generator

Function	Produces structured guidance from RiskAssessment
Output fields	what_happened (string), why_risky (string), what_to_do (ordered string list)
Source	Derived from evidence_collection + threat_detection type via templated + LLM-assisted generation

5. Data Contracts (Schemas)

5.1 Request Payload

```
{
  "input_type": "screenshot | text | url | file",
  "payload": "",
  "user_preferences": {
    "who_are_you": "student | professional | personal_user",
    "technical_level": "simple | detailed | technical",
    "focus_area": ["emails_messages", "websites_links", "account_security",
                  "device_security", "network_security", "everything"]
  }
}
```

5.2 InputProcessorOutput

```
{
  "normalized_text": "string",
  "extracted_urls": ["string"],
  "sender_info": { "name": "string", "email_or_domain": "string" },
  "ssl_info": { "valid": true, "issuer": "string" },
  "domain_age_days": 12,
  "low_confidence": false
}
```

5.3 RiskAssessment (Risk Engine Output)

```
{
  "risk_level": "low | medium | high",
  "confidence": 0.94,
  "category": "phishing | malware | malicious_url | social_engineering |
             account_security | device_security | network_threat | web_security",
  "signals": ["urgent_language", "suspicious_link", "sender_mismatch"],
  "evidence_collection": [
    { "reason": "Suspicious link not from official domain", "source": "rules_engine" },
    { "reason": "Domain registered 3 days ago", "source": "security_checks" }
  ],
  "extracted_url": "http://bit.ly/xyz123"
}
```

5.4 Final Response (Analysis Output)

```
{
  "risk_assessment": { ...RiskAssessment... },
  "explanation": {
    "simple": "string",
    "detailed": "string",
    "technical": "string"
  },
  "why_suspicious": ["string"],
  "what_could_happen": "string",
  "action_plan": {
    "what_happened": "string",
    "why_risky": "string",
    "what_to_do": ["string"]
  }
}
```

6. API Specification

Method	Endpoint	Description
POST	/api/v1/analyze	Submit content (screenshot/text/url/file) + preferences; returns full Analysis Output
GET	/api/v1/analyze/{id}	Retrieve a previously generated analysis result by ID
POST	/api/v1/preferences	Save/update a user's default preferences (who_are_you, technical_level, focus_area)
GET	/api/v1/threat-categories	Returns the fixed list of 8 supported threat categories

7. Risk Scoring Logic

The Risk Engine combines three inputs into one severity verdict:

```
risk_score = (w1 * rules_engine_score)
             + (w2 * ml_model_probability)
             + (w3 * failed_security_checks_ratio)

severity:
  risk_score >= 0.75 → High
  0.40 <= risk_score < 0.75 → Medium
  risk_score < 0.40 → Low

confidence = model + rule agreement factor (higher when Rules Engine
                                             and ML Model agree on the same category)
```

Weights (w1, w2, w3) are configurable and tuned against a labeled validation set.

8. Non-Functional Requirements

Requirement	Definition
Latency	End-to-end analysis result returned in under 8 seconds for a single submission
Availability	99.5% uptime for the analysis API
Scalability	Security Engines run in parallel; horizontally scalable stateless API workers
Accuracy	Target $\geq 90\%$ precision on the phishing/malicious-URL detection task before production release
Usability	Analysis Output must render with zero technical jargon in the Simple explanation style
Auditability	Every risk verdict must be traceable to its evidence_collection entries

9. Security & Privacy Requirements

- Uploaded screenshots/files are stored only as long as needed to complete analysis, then purged on a defined retention schedule.
- All payloads (screenshot, text, URL, file) are transmitted over TLS.
- PII extracted during analysis (emails, names) is not persisted beyond the analysis record unless the user opts in to history.
- No user-submitted content is used to retrain models without explicit consent.
- Access to stored analysis results is authenticated and scoped to the submitting user.

10. Error Handling

Condition	Handling
Unreadable / blurry screenshot	Return low_confidence flag; prompt user to re-upload
Unsupported file type	Reject with a clear supported-formats message
URL unreachable	Fall back to lexical/domain-only analysis; flag as partial_analysis
ML Model or LLM service timeout	Retry once; on failure, degrade to Rules Engine + Security Checks only, flagged as reduced_confidence
Ambiguous verdict (score near threshold)	Default to the higher severity category (fail-safe toward caution)

11. Deployment Architecture

- Containerized microservices: API Gateway, Analysis Orchestrator, Security Engines, LLM Explanation Layer.
- Vision/OCR and LLM Explanation calls made to Claude via API (no self-hosted LLM required for MVP).
- ML Model served as an internal inference endpoint, versioned independently from the API.
- Relational DB for analysis records; object storage for temporary uploaded content.
- Environment separation: development, staging, production, each with isolated credentials and data.